The Relay

Assignment brief

A company runs an assistant that answers engineers' questions about a small library of research papers, by chaining several model calls and a few tools. It works out what the person is asking, retrieves passages from the papers, decides what to do, drafts a cited reply, and checks the reply before sending it. On a quiet afternoon it feels fast. At launch it will not.

Requests arrive in bursts, usually right after something goes wrong for many users at once. Most requests need a few steps. Some need many. A few loop. The model provider limits requests per minute, tokens per minute and requests per day. The limits differ by plan and by model, and they can change without notice. Tools time out. Some calls are slow for no visible reason. Some calls succeed slowly instead of failing quickly. People are waiting, and they give up sooner than you expect.

The company wants every reply fast, every reply good, the bill small, and nobody turned away. What "fast" means is up to you. It cannot have all four. It also never wants to send a wrong or embarrassing answer just to stay fast.

We left out the details on purpose, and some of the goals pull against each other. Decide what matters, choose your own numbers, and make the calls a real engineer would have to make. Where you give up one goal for another, say so.

What to build

Build a small working version of the pipeline in Python, C++ or Go (your choice). It needs at least four steps, at least one tool call, and at least one branch or loop. One step must retrieve passages from the library below.

The library

The assistant answers only from these three papers on arXiv. Fetch them yourself, at exactly these versions:

- "Attention Is All You Need", arXiv 1706.03762, version 7
- "Moshi: a speech-text foundation model for real-time dialogue", arXiv 2410.00037, version 1
- "Dynamic Chunking for End-to-End Hierarchical Sequence Modeling", arXiv 2507.07955, version 2

Every answer must cite where in the papers it came from. If the papers do not say, the assistant must say so instead of guessing. We will ask questions you cannot answer from memory, and some that the papers do not answer at all.

Use a real hosted model on Groq's free tier for the model calls, with your own free account and key. Its limits are real, so find out what they are from its documentation and from what it sends back when you reach one. A free tier will not survive heavy load, so you will also write a stand-in model service you control. It must be able to be slow, to fail, to refuse because a limit was reached, to become slower under load, and to send its reply a piece at a time. Calibrate it from what you measured on the real service, and say where it is least like the real thing.

What to run

First, send a modest number of real requests to learn how the real service behaves: how its latency is spread, what its limits are, and what it does when one is reached. Then run your pipeline under at least three load shapes of your own design, for example steady, a sudden burst, and sustained overload. Use the stand-in for loads the real service cannot take. Keep the raw results for both. We will look at them.

What your 2 pages must contain

Use these headings in this order. A page that skips one is incomplete.

1. **What I built and why.** The design, and the one decision that shaped it most.
2. **The real service and my stand-in.** The limits you found in the real service and how you found them. How your stand-in behaves, how you calibrated it from real measurements, and where it differs most from the real thing.
3. **Results.** Time to first token and total time, each at the median and at the slow end (say which percentiles and how many requests), failures, and given-up requests, for each load shape. Say which runs were real and which used the stand-in. Put every number in the results tables below. Include a number we could not have guessed without running it. Also report answer quality on at least 15 questions you wrote yourself, and tokens used per answer.
4. **Where the time goes.** For one request that was slow, split its latency into waiting, working and retrying, and say whether the delay came before or after the first token. Show how you know.
5. **When a limit is hit.** What your system does at the per-minute limit and at the per-day limit. Who is delayed, who is dropped, who gets a weaker answer, and why.
6. **Retries.** Your policy, and one case where retrying made things worse.
7. **What the person sees.** At 3pm the daily budget is gone. Describe the reply a waiting person receives.
8. **What I'm unsure about, what I left out, what I learned.** Your least certain assumption and what breaks if it is wrong. Two things you deliberately did not build. One thing that was new to you, taught to us in your own words.

Add two more headings after heading 5:

Retrieval: quality versus cost. Try at least three settings for how you split the papers and how much you fetch (piece size, number of pieces, and any re-ranking). For each, report answer quality, tokens per answer, and time to first token. Choose one setting and say why, and say what you do with retrieval when the limit is close.

How this library will mislead a naive retriever. Name three ways, each with an example question where it goes wrong.

One more heading: Your feedback on Vaani. Visit https://www.vaani.dev/ and tell us what you honestly think, in a few lines: what stood out, what confused you, and one thing you would change. There is no right answer.

Finish with a section called How I worked with the assistant. Include your real conversation history with it, not a cleaned-up one, or the prompts if the tool cannot export. Then, in half a page, give the three prompts that mattered most and why, one thing it got wrong that you caught and how you caught it, and one decision where you overruled it. Say what in this submission is distinctly yours. There is no single correct answer. We care whether your choices follow from your measurements and whether you can say what they cost.

The results tables (required)

Every number goes in these tables, completely filled in. Write "not measured" where you did not measure something. Reviewers read these first and recompute them from your raw file, so they must match. Use milliseconds for time and one decimal for percentages. Copy the tables into your write-up.

Table 1: Limits you found in the real service

Limit	Value you found	How you found it	Confidence (high, medium, low)
Requests per minute			
Tokens per minute			
Requests per day			
Tokens per day			
Requests in flight at once			
What it returns when a limit is hit			
Time until it recovers			

Table 2a: Latency, one row per load scenario (at least: steady, burst, sustained overload, per-minute limit halved, daily budget exhausted)

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)

Table 2b: Outcomes, same rows

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers

Table 3: Retrieval settings (at least three)

Setting (piece size, pieces fetched, re-rank yes or no)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)

Raw file. Submit one file named results.csv with one row per request and these columns: request_id, scenario, source (real or stand-in), start_time, first_token_ms, total_ms, status (ok, rate_limited, timeout, error), retries, tokens_in, tokens_out, retrieval_setting, answer_grade (correct, wrong, refused_correctly, refused_wrongly). Every figure in the tables must be computable from this file.

Ground rules

- Submit within 4 days of receiving this brief. We are not scoring who spent the most time.
- Maximum 2 pages of writing, plus your code, raw results and your conversation history with the assistant.
- Use your own free account. Do not include your key in your submission. Your account's daily limit is shared by everything you run, so plan your requests: running out is part of the problem.
- Stay within the provider's terms. Do not use extra accounts or any other trick to get around its limits. Working inside the limits is the point.
- **We expect most of you will build this with an AI coding assistant, and that is welcome. What we are looking at is how well you direct it: how you frame the problem, how you check what it gives you, and where you overrule it. We are also looking for you in the submission: the choices, measurements and reasoning that someone else's prompt would not have produced.** A short live conversation follows where you explain your results and change your system, so only submit what you can defend.
- You do not need to cover every technique. Pick the few that matter for your measurements and go deep.